

ELEC 475: Lab 2 — In-Lab Manual

Building & Modernizing LeNet-5 (2-Hour Hands-On Session)

Course: ELEC 475 — Computer Vision with Deep Learning (Fall 2026)

Department: Electrical & Computer Engineering, Queen's University

Instructor: Dr. Michael Greenspan

Lab Developer & Teaching Assistant: Arshia Esmail Tehrani (hd53@queensu.ca)

Teaching Team: Yawar Ashraf, Connor Johnston, Tyler Wilson, Naser Al-Obeidat

Duration: Strictly 2 Hours (120 Minutes)

Deliverables: In-lab TA Checkoff (3 Marks)

Table of Contents

1. The 3-Phase Lab 2 Agenda
2. Session Overview & 2-Hour Time Budget
3. Step 1: Environment Activation & Project Tour (15 Minutes)
4. Step 2: Implement Base LeNet-5 for RGB 64×64 (35 Minutes)
 - 2.1 Architecture Specification & Mermaid Diagram
 - 2.2 Layer Dimensions & Spatial Math Walkthrough
 - 2.3 Code Implementation & Parameter Verification
5. Step 3: Train Baseline LeNet on CIFAR-10 (15 Minutes)
6. Step 4: Implement Modernized LeNet (35 Minutes)
 - 4.1 Ablation 1: ReLU Activations
 - 4.2 Ablation 2: Overlapping Max Pooling
 - 4.3 Modernized Architecture Specification & Mermaid Diagram
 - 4.4 Spatial Dimension Calculations
7. Step 5: Train Modernized LeNet & Observe Convergence Dynamics (10 Minutes)
8. Step 6: In-Lab TA Evaluation & Checkoff (10 Minutes)
 - 6.1 Checkoff Criteria
 - 6.2 In-Lab Marking Rubric (3 Marks Total)

1. The 3-Phase Lab 2 Agenda

1. Pre-Lab - Preparation Before Week 4

Task 1: Environment Isolation (.venv & pip install)



Task 2: Dataset Download (CIFAR-10 & Tiny ImageNet)



Task 3: Exploratory Data Analysis (utils/eda.py)



Task 4: Automated Verification (prelab_verify.py)



2. In-Lab - 2-Hour Hands-On Milestone

Step 1: Environment Activation and Project Tour - 15 min



Step 2: Implement Base LeNet-5 (64x64x3) in models/lenet.py - 35 min



Step 3: Train Baseline LeNet (5 Epochs) on CIFAR-10 - 15 min



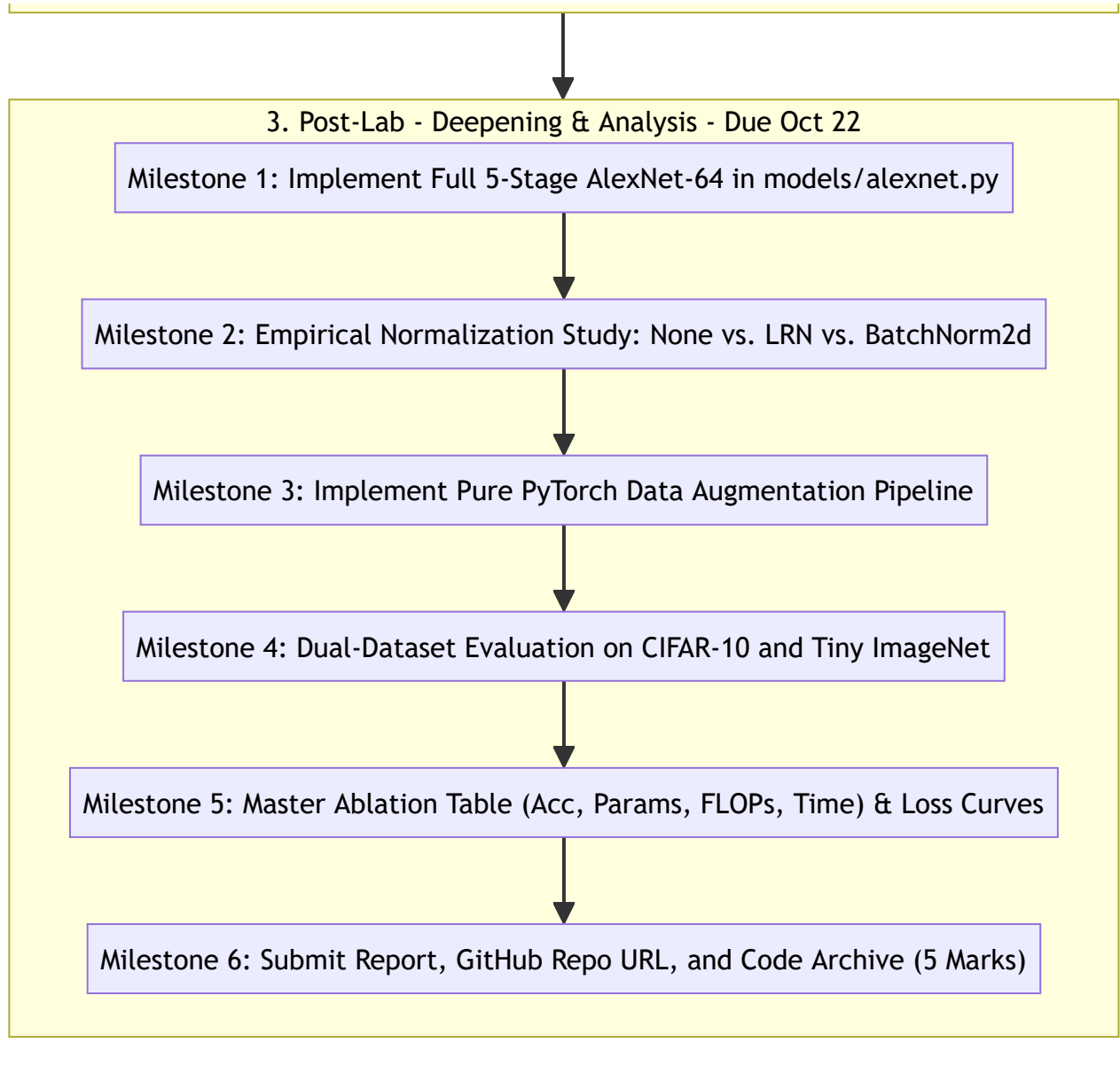
Step 4: Implement Modernized LeNet (ReLU + Overlapping MaxPool) - 35 min



Step 5: Train Modernized LeNet and compare convergence dynamics - 10 min



Step 6: In-Lab TA Checkoff (3 Marks) - 10 min



2. Session Overview & 2-Hour Time Budget

This 2-hour hands-on session is tightly scheduled to ensure you build, debug, and train two distinct convolutional architectures and observe the immediate performance impact of modern activations and pooling.

Time Allocation	Activity	Key Deliverable
00:00 – 00:15 (15 min)	Step 1: Project Tour & Virtual Environment Activation	Working terminal & dataset check
00:15 – 00:50 (35 min)	Step 2: Implement Base LeNet-5 in <code>models/lenet.py</code>	Verified parameter count via <code>torchinfo</code>
00:50 – 01:05 (15 min)	Step 3: Train Baseline LeNet (5 Epochs)	Baseline loss curve & validation metrics

Time Allocation	Activity	Key Deliverable
01:05 – 01:40 (35 min)	Step 4: Implement Modernized LeNet in <code>models/lenet_modern.py</code>	Ablation 1 (ReLU) & Ablation 2 (Overlapping MaxPool)
01:40 – 01:50 (10 min)	Step 5: Train Modernized LeNet (5 Epochs)	Modernized loss curve & comparative metrics
01:50 – 02:00 (10 min)	Step 6: TA Checkoff & Evaluation	In-lab grade recorded (3 Marks)

Step 1: Environment Activation & Project Tour (15 Minutes)

1. Open your terminal (PowerShell on Windows, or bash in WSL/macOS).
2. Navigate to your lab starter workspace:

```
cd path/to/lab2_template
```

3. Activate your virtual environment:

```
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Windows Command Prompt (CMD):
.\.venv\Scripts\activate.bat

# macOS / Linux / WSL:
source .venv/bin/activate
```

4. Verify the modular directory structure:

```

lab2_template/
├── configs/
│   └── config.py          # Centralized hyperparameter and path definitions
├── models/
│   ├── __init__.py
│   ├── lenet.py           # TODO (In-Lab): Base LeNet-5 implementation
│   ├── lenet_modern.py    # TODO (In-Lab): Modernized LeNet (ReLU + MaxPool)
│   ├── alexnet.py         # TODO (Post-Lab): Full AlexNet implementation
│   └── zoo.py              # PyTorch Hub & torchvision reference architectures
├── utils/
│   ├── __init__.py
│   ├── dataset.py         # Automated CIFAR-10 and Tiny ImageNet loaders
│   ├── eda.py             # Exploratory Data Analysis tool (with imbalance diagnostics)
│   ├── transforms.py      # PyTorch data augmentation pipeline
│   ├── metrics.py         # Multi-metric evaluation (Accuracy, Macro/Micro F1) & EarlyStopping
│   ├── profiler.py        # Model complexity profiler (Params, MACs, FLOPs)
│   ├── seed.py            # Global stochastic seeding and Bessel-corrected stats
│   └── visualization.py   # Training curves and sample grid plotters
├── train.py               # Complete CLI training engine with --seed and --early-stopping support
├── evaluate.py            # Model evaluation script
├── compare_models.py      # Multi-model complexity and FLOPs benchmarking suite
├── demo_transforms.py     # Pre-Lab transform benchmark script
└── requirements.txt

```

5. Confirm that `torch` and `torchvision` are accessible:

```
python -c "import torch, torchvision; print('PyTorch OK! CUDA:', torch.cuda.is_available())"
```

Step 2: Implement Base LeNet-5 for RGB 64×64 (35 Minutes)

You will now implement the classical **LeNet-5** (LeCun et al., 1998) adapted for 3-channel RGB input at 64×64 spatial resolution.

2.1 Architecture Specification & Mermaid Diagram

Base LeNet-5 Architecture - models/lenet.py

Input Image: (3, 64, 64) RGB



Conv2D (5x5, s=1, p=0, 6 filters) - Output: (6, 60, 60)



Tanh Activation Function



Average Pooling (2x2, s=2) - Output: (6, 30, 30)



Conv2D (5x5, s=1, p=0, 16 filters) - Output: (16, 26, 26)



Tanh Activation Function

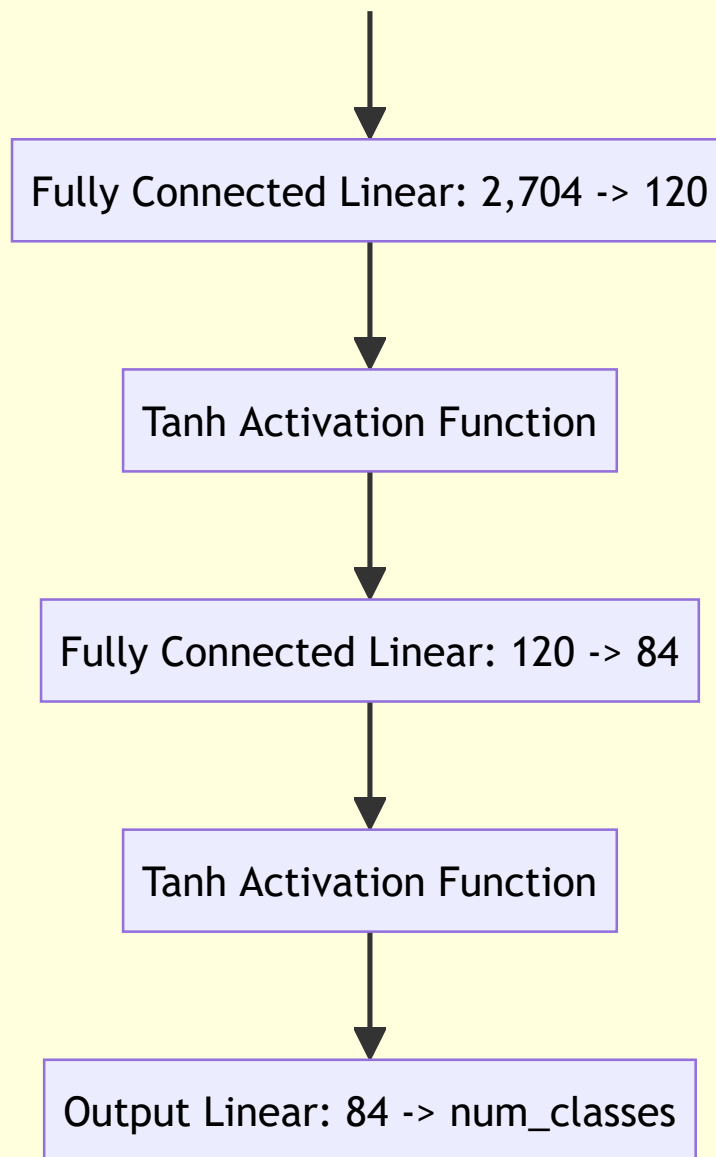


Average Pooling (2x2, s=2) - Output: (16, 13, 13)



Flatten Layer: $16 \times 13 \times 13 = 2,704$ features





2.2 Layer Dimensions & Spatial Math Walkthrough

Using the spatial formula ($N_o = \lfloor \frac{N_I - K + 2P}{S} \rfloor + 1$):

- **Input:** (3, 64, 64)
- **conv1:** `nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0)`
 $\Rightarrow N_o = \frac{64-5+0}{1} + 1 = 60 \Rightarrow (6, 60, 60)$
- **Activation:** `nn.Tanh()`
- **pool1:** `nn.AvgPool2d(kernel_size=2, stride=2)`
 $\Rightarrow N_o = \frac{60-2}{2} + 1 = 30 \Rightarrow (6, 30, 30)$
- **conv2:** `nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)`
 $\Rightarrow N_o = \frac{30-5+0}{1} + 1 = 26 \Rightarrow (16, 26, 26)$
- **Activation:** `nn.Tanh()`
- **pool2:** `nn.AvgPool2d(kernel_size=2, stride=2)`
 $\Rightarrow N_o = \frac{26-2}{2} + 1 = 13 \Rightarrow (16, 13, 13)$

- **Flatten:** $16 \times 13 \times 13 = \mathbf{2,704}$ features
- **fc1:** `nn.Linear(in_features=2704, out_features=120) → nn.Tanh()`
- **fc2:** `nn.Linear(in_features=120, out_features=84) → nn.Tanh()`
- **fc3:** `nn.Linear(in_features=84, out_features=num_classes)`

2.3 Code Implementation & Parameter Verification

Open `models/lenet.py` and implement the `__init__` and `forward` methods of the `LeNet5` class.

Then run the model inspection test:

```
python -m models.lenet
```

The script will display the layer summary using `torchinfo` and calculate model complexity via `ModelProfiler`:

- **Total Trainable Parameters:** Architectural baseline scale ~ 0.34 M (approx. 3.38×10^5 parameters for 10 output classes). Verify your tensor dimensions and layer configurations if your parameter magnitude deviates.
- **Computational Heaviness (FLOPs / MACs):** Nominal complexity scale ~ 2.4 M MACs (~ 4.8 MFLOPs for $64 \times 64 \times 3$ input). Record the exact parameter count, MACs, and FLOPs reported by your profiler output.

Step 3: Train Baseline LeNet on CIFAR-10 (15 Minutes)

Now execute the training script for 5 epochs. The training engine handles optimization (Adam, $lr = 10^{-3}$), `CrossEntropyLoss`, automated validation, and wall-clock execution timing via our synchronized `Timer`:

```
python train.py --model lenet --dataset cifar10 --epochs 5 --batch-size 128 --seed 42 --save-model c
```

Expected Output & Observations:

- **Reproducibility & Model Profiler Output:** `train.py` reports the active seed and theoretical complexity at startup:

```
[Reproducibility] Global Seed: 42 (Deterministic cuDNN algorithms enabled)
[Model Profiler] Trainable Params: 338,486 | MACs: 2.41 M | FLOPs: 4.82 M
```

- **Real-Time Epoch Timing & Multi-Metric Logging:** Each epoch logs true hardware wall-clock time, loss, accuracy, and Macro F_1 :

```
Epoch 01/05 Summary: Train Loss: 1.6214, Train Acc: 42.10% | Val Loss: 1.4120, Val Acc: 49.30% |
```

- **Training Time:** $\approx 2.5\text{--}3$ minutes on a GPU (or $\approx 6\text{--}8$ minutes on a multi-core CPU).
- **Validation Accuracy Observation:** In standard 5-epoch training on CIFAR-10, baseline LeNet typically settles in the mid-to-high 50s to low 60s (%), with exact convergence depending on pseudo-random initial

weight conditions. Record your empirical validation accuracy and Macro F_1 .

- Notice that Tanh exhibits relatively slow initial convergence due to saturation at the tails $([-1, 1])$.
- Inspect the generated plot in `plots/loss_lenet.png`.

Step 4: Implement Modernized LeNet (35 Minutes)

We will now perform the first two critical steps of our **Ablation Study**, introducing the architectural upgrades that bridge LeNet to AlexNet:

4.1 Ablation 1: ReLU Activations

Replace `nn.Tanh()` with `nn.ReLU()`.

- *Theoretical Reason:* As discussed in Lecture 05, ReLU ($f(x) = \max(0, x)$) does not saturate for positive activations, providing constant gradient 1.0 and completely preventing vanishing gradients during backpropagation.

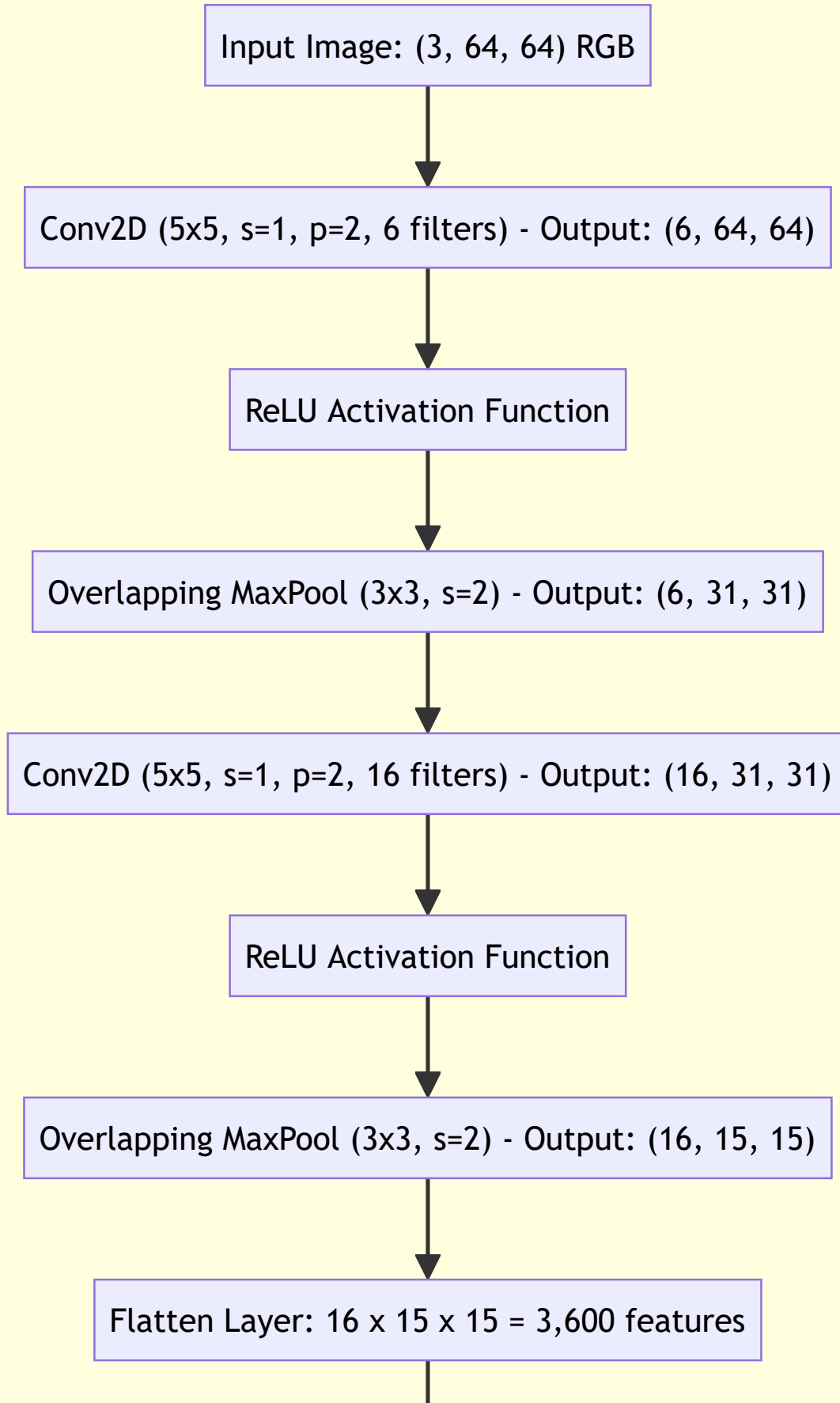
4.2 Ablation 2: Overlapping Max Pooling

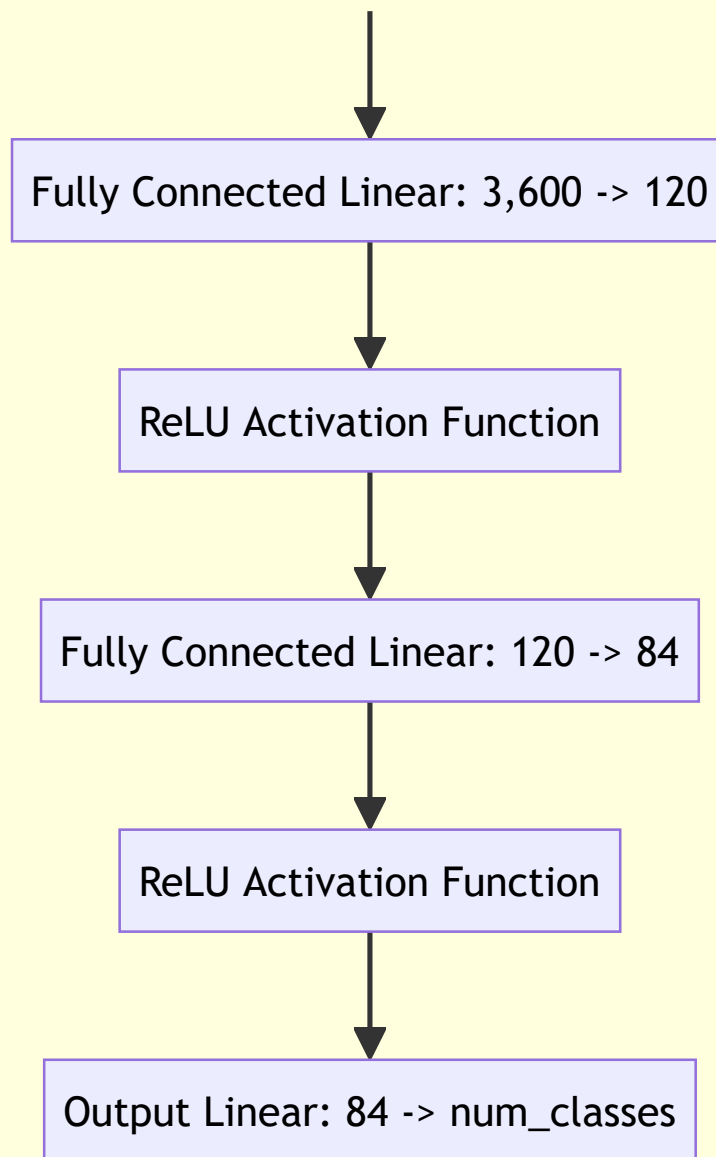
Replace non-overlapping Average Pooling with **Overlapping Max Pooling** ($K = 3, S = 2$, as seen in Lecture 05 Slide 7).

- *Theoretical Reason:* Max pooling preserves dominant edge activations rather than blurring them. Overlapping strides ($S < K$, e.g., stride 2 with kernel 3) reduce feature map spatial dimensionality while helping prevent spatial overfitting.

4.3 Modernized Architecture Specification & Mermaid Diagram

Modernized LeNet Architecture - models/lenet_modern.py





4.4 Spatial Dimension Calculations

To accommodate overlapping pooling while preserving clean spatial dimensions, we apply padding $P = 2$ to the convolutions:

- **Input:** $(3, 64, 64)$
- **conv1:** `nn.Conv2d(3, 6, kernel_size=5, stride=1, padding=2)` $\implies (6, 64, 64)$
- **Activation:** `nn.ReLU()`
- **pool1 (Overlapping MaxPool):** `nn.MaxPool2d(kernel_size=3, stride=2)`
 $\implies N_o = \lfloor \frac{64-3}{2} \rfloor + 1 = 31 \implies (6, 31, 31)$
- **conv2:** `nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=2)` $\implies (16, 31, 31)$
- **Activation:** `nn.ReLU()`
- **pool2 (Overlapping MaxPool):** `nn.MaxPool2d(kernel_size=3, stride=2)`
 $\implies N_o = \lfloor \frac{31-3}{2} \rfloor + 1 = 15 \implies (16, 15, 15)$
- **Flatten Size:** $16 \times 15 \times 15 = \mathbf{3,600}$ features

- `fc1 : nn.Linear(3600, 120) → nn.ReLU()`
- `fc2 : nn.Linear(120, 84) → nn.ReLU()`
- `fc3 : nn.Linear(84, num_classes)`

Open `models/lenet_modern.py` and complete the `LeNetModern` class. Verify dimensions and FLOPs:

```
python -m models.lenet_modern
```

Notice the comparative scaling: Trainable parameters increase moderately to $\sim 0.45 \text{ M}$ ($\approx 4.46 \times 10^5$), and computational complexity scales to $\sim 4.1 \text{ M MACs}$ ($\sim 8.2 \text{ MFLOPs}$) due to the larger flattened dense feature vector after overlapping pooling. Record the exact values reported by your profiler.

Step 5: Train Modernized LeNet & Observe Convergence Dynamics (10 Minutes)

Execute training for 5 epochs with identical hyperparameters:

```
python train.py --model lenet_modern --dataset cifar10 --epochs 5 \
  --batch-size 128 --save-model checkpoints/lenet_modern.pth --plot-file plots/loss_lenet_modern.p
```

Empirical Comparison & Convergence Dynamics

Observe the output in your terminal and compare your two training runs:

- **Baseline LeNet (Tanh + AvgPool):** Note your final validation loss, validation accuracy, Macro F_1 , and measured FLOPs after 5 epochs.
- **Modernized LeNet (ReLU + Overlapping MaxPool):** Note your final validation loss, validation accuracy, Macro F_1 , and measured FLOPs after 5 epochs.

Analytical Questions for Discussion & Checkoff:

1. Performance Improvement Analysis:

- Did your Modernized LeNet achieve higher validation accuracy and Macro F_1 than the baseline within 5 epochs? (Yes / No).
- **If Yes:** Explain the underlying architectural and mathematical reasons. How did substituting saturating Tanh with non-saturating ReLU activations eliminate vanishing gradients, and how did overlapping max pooling preserve salient localized edge features?
- **If No or Marginal:** Inspect your training and validation loss curves (`plots/loss_lenet.png` vs. `plots/loss_lenet_modern.png`). What experimental factors (e.g., initial learning rate, short 5-epoch training window, or stochastic variance) might account for the lack of separation?

2. Computational Speed & FLOPs Trade-Off Analysis:

- Did the modernized architecture require higher computational resources (FLOPs / epoch execution time)? (Yes / No).
- Report your observed speed and throughput: Did the increase in arithmetic FLOPs ($\sim 4.8 \text{ M} \rightarrow \sim 8.2 \text{ MFLOPs}$) noticeably slow down epoch wall-clock seconds or batch throughput on your hardware?
- **Is the heavier model trade-off worthwhile?** Where is Modernized LeNet clearly better (e.g., substantially faster convergence and higher classification accuracy with negligible latency penalty on modern GPUs), and where might the increased FLOPs and parameter footprint be a disadvantage (e.g., ultra-low-power edge microcontrollers with strict SRAM limits)?
- **Modern Hardware Context:** Why do small arithmetic FLOP differences matter far less on parallel modern GPUs with Tensor Cores (which thrive on saturating compute pipelines) than on sequential CPU or edge microcontroller cores?

3. Early Stopping & Trajectory Dynamics:

- Observe the validation loss trajectories over the 5 epochs. Has the validation loss plateaued or is it continuing to descend? Based on this trajectory, would an Early Stopping controller halt training or allow it to proceed further?

Step 6: In-Lab TA Evaluation & Checkoff (10 Minutes)

6.1 Checkoff Criteria

To receive full marks for your in-lab component, demonstrate the following to a TA:

1. Show your completed `models/lenet.py` and `models/lenet_modern.py` code.
2. Show the terminal output confirming successful execution of both training runs.
3. Show the generated loss curves: `plots/loss_lenet.png` and `plots/loss_lenet_modern.png`.
4. Be prepared for a brief oral checkoff with your TA: explain the spatial dimension math leading to the dense layer input shapes ($16 \times 13 \times 13 = \mathbf{2,704}$ in LeNet vs. $16 \times 15 \times 15 = \mathbf{3,600}$ in Modernized LeNet), justify why ReLU converges faster than Tanh, and analyze your observed convergence dynamics.

6.2 In-Lab Marking Rubric (3 Marks Total)

Evaluation Item	Criteria	Marks
Base LeNet-5 Implementation	Clean <code>models/lenet.py</code> , correct dimensions, parameter summary verified.	0.5
Baseline Training Run	Successful execution of <code>train.py --model lenet</code> , valid <code>loss_lenet.png</code> .	0.5
Modernized LeNet Implementation	Correct implementation of ReLU and Overlapping MaxPool in <code>models/lenet_modern.py</code> .	1.0

Evaluation Item	Criteria	Marks
Ablation Comparison & Oral Defence	Clear demonstration to TA of observed convergence dynamics, accuracy changes, and mathematical derivation of layer dimensions.	1.0
Total In-Lab Mark		3.0

The remaining 4 marks of Lab 2 will be completed as part of the Post-Lab, due October 22, 2026 before 23:59 ET.

 TIP: Completed Checkoff Early? Leverage In-Person TA Support!

If you complete your in-lab checkoff before the 2-hour session concludes, do not leave early! Take full advantage of having TAs present in the room to assist with architecture design and debugging. We strongly encourage you to immediately transition to **Post-Lab Milestone 1** (implementing Full AlexNet-64 in `models/alexnet.py`) and initiate your **Normalization Study** (`POSTLAB.md` Step 2). Building AlexNet and resolving tensor shape questions while TAs are in the room will save you significant time during independent work.